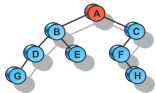
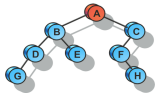


Хеш-таблицы с открытой адресацией



Хеш-таблица с открытой адресацией

- ▶ Все элементы хранятся в таблице
 - ▶ Нет лишней траты памяти на указатели
 - ▶ Ячейка таблицы свободна, если там хранится NIL
- ▶ В зависимости от элемента выбирается порядок ячеек таблицы
- ▶ Формально, можно сказать, что теперь хеш-функция от двух аргументов
 - ▶ $h(x, i)$ — в какую ячейку надо посмотреть для элемента x на i -м шаге поиска



Вставка

- ▶ Ячейки просматриваются одна за другой в таком порядке до первой свободной
- ▶ Элемент добавляется в первую найденную свободную ячейку
- ▶ Hash-Insert(T, x)

$i \leftarrow 0$

repeat

$j \leftarrow h(x, i)$

if $T[j] = \text{NIL}$ **then**

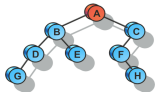
$T[j] \leftarrow x$

return j

else

$i \leftarrow i + 1$

until $i = m$



Поиск

- ▶ При поиске ячейки просматриваются в том же порядке
 - ▶ Если пришли к NIL, то можем быть уверены, что элемента нет
 - ▶ Пока из таблицы никакие элементы не удаляются
- ▶ Hash-Search(T, x)

$i \leftarrow 0$

repeat

$j \leftarrow h(x, i)$

if $T[j] = x$ **then**

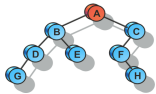
return j

else

$i \leftarrow i + 1$

until $T[j] = \text{NIL}$ или $i = m$

return NIL



Удаление

- ▶ Удаление нельзя произвести просто очисткой ячейки, которая занята удаляемым элементом
 - ▶ Для некоторого другого элемента можем закончить поиск раньше, чем найдем его
- ▶ Заведем отдельную пометку `DELETED`, которая будет означать, что в этой ячейке был элемент
- ▶ При поиске будем эту пометку пропускать и идти дальше
- ▶ При вставке будем ее перезаписывать, если не нашли
 - ▶ Нужно сначала сделать поиск до первого `NIL`
 - ▶ Иначе элемент будет добавлен дважды